LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA

PRAKTIKUM 2

Dzakiy Ibnu Khoir (5225600061)

PROGRAM STUDI SARJANA TERAPAN TEKNOLOGI GAME

JURUSAN TEKNOLGOI MULTEMDIA KREATIF

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2026

Latihan 1

1. **Code**

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

int main() {
    Node* node1 = new Node;
    Node* node2 = new Node;
    Node* node3 = new Node;

    node1->data = 10;
    node2->data = 20;
    node3->data = 30;

    node1->next = node2;
    node2->next = node3;
    node3->next = nullptr;

    Node* temp = node1;
    while (temp != nullptr){
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "Null\n";

}
```

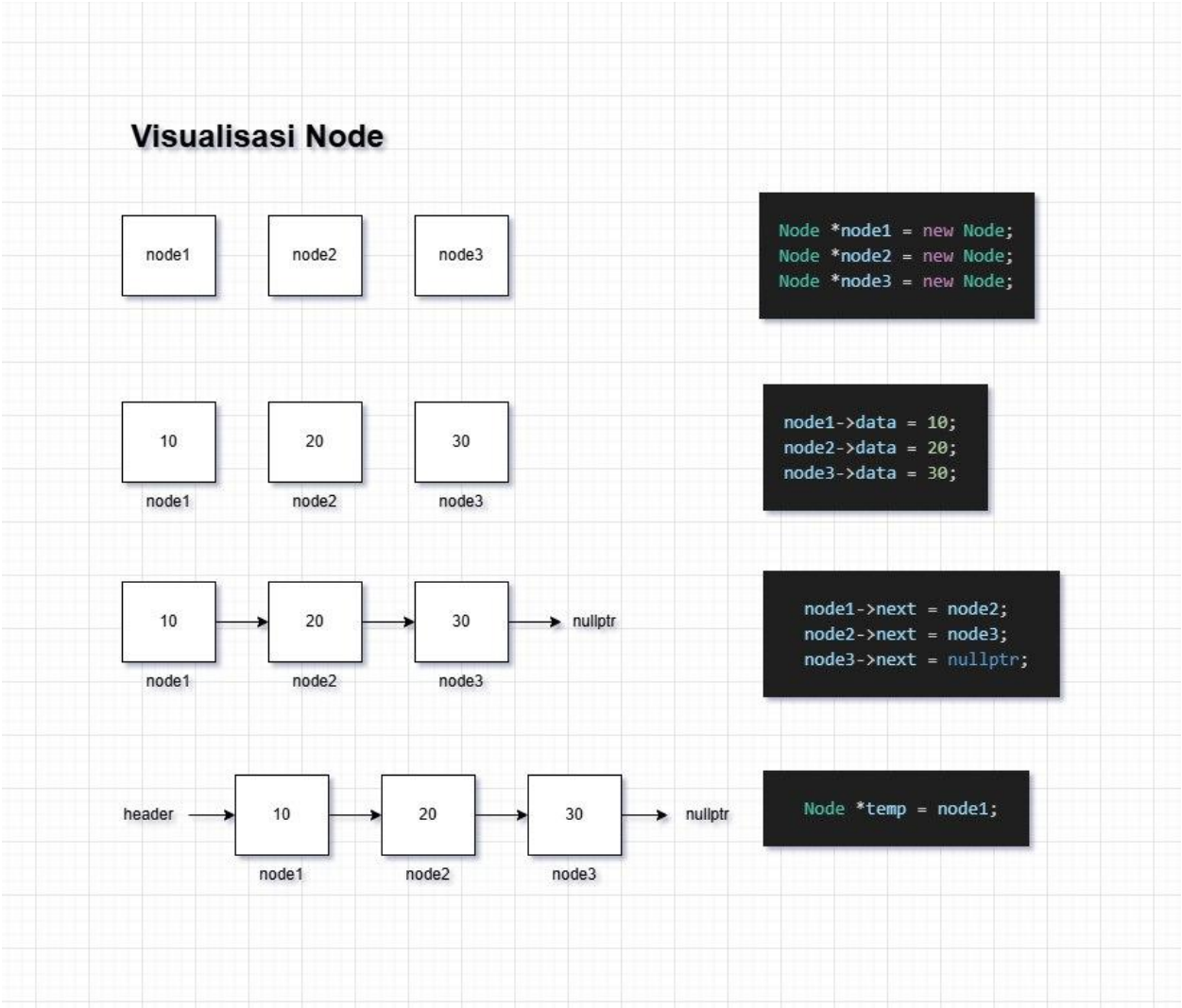
2. **Output**

```
e }
10 -> 20 -> 30 -> Null
```

3. **Penjelasan**

Program tersebut membuat tiga node secara dinamis, menghubungkannya menjadi sebuah singly linked list berisi 10 → 20 → 30, lalu melakukan traversal untuk menampilkan seluruh isi list hingga mencapai nullptr.

4. **Visualisasi**



Latihan 2

1. **Code**

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *next;
};

Node *head = nullptr;

void insertFront(int value)
```

```
{
    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = head;
    head = newNode;
}

int main()
{
    Node *node1 = new Node{50, nullptr};
    Node *node2 = new Node{60, nullptr};
    Node *node3 = new Node{70, nullptr};

    head = node1;

    node1->next = node2;
    node2->next = node3;
    node3->next = nullptr;


    insertFront(30);
    insertFront(20);
    insertFront(10);

    Node *temp = head;
    while (temp != nullptr)
    {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "Null\n";
}
```

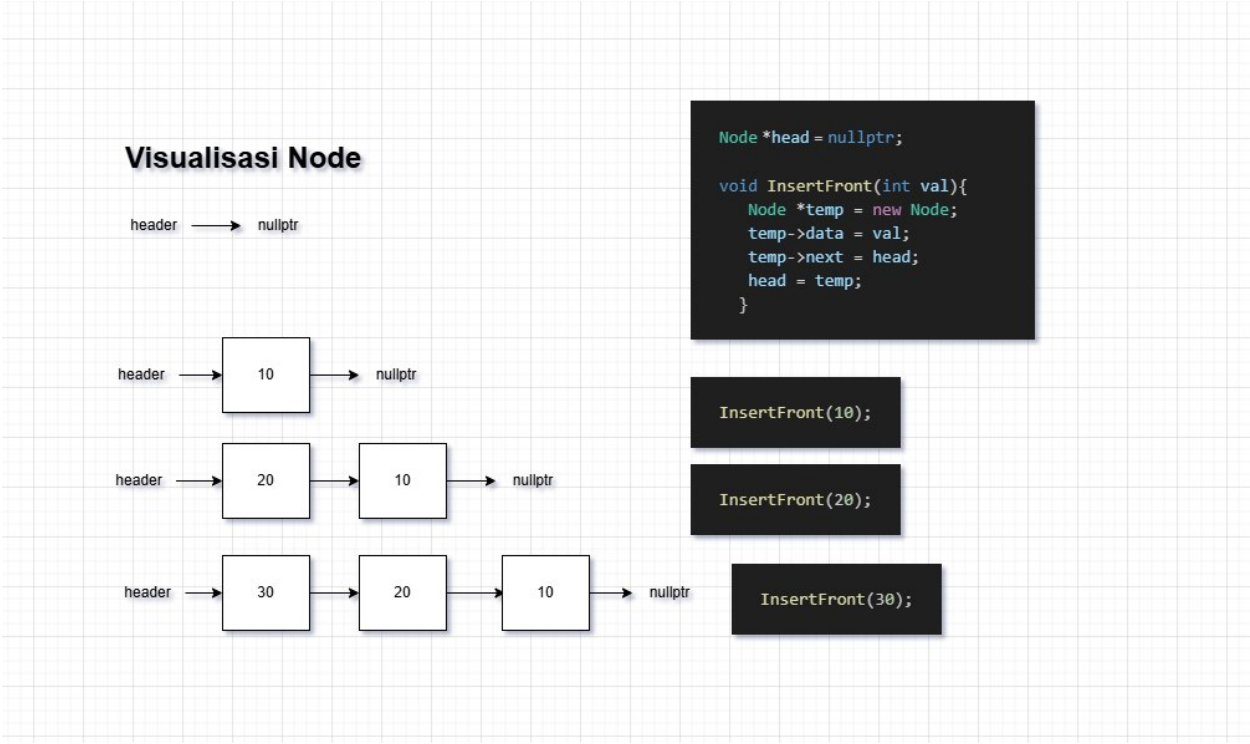
2. Output

```
\latihan2 }
10 -> 20 -> 30 -> 50 -> 60 -> 70 -> Null
PS E:\DokumenKuliah\Semester 2\Algoritma\P
```

3. Penjlasan

Program tersebut membuat linked list awal berisi 50 → 60 → 70, lalu menambahkan tiga node baru (10, 20, 30) di bagian depan menggunakan fungsi insertFront, sehingga akhirnya menjadi 10 → 20 → 30 → 50 → 60 → 70, kemudian menampilkan seluruh isi list hingga nullptr

4. Visualisasi



Latihan 3

1. **Code**

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *next;
};

Node *head = nullptr;

void insertBack(int value)
{
    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = nullptr;
    if (head == nullptr)
    {
        head = newNode;
        return;
```

```
    }
    Node *temp = head;
    while (temp->next != nullptr)
        temp = temp->next;
    temp->next = newNode;
}

void insertFront(int value)
{
    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = head;
    head = newNode;
}

int main()
{
    Node *node1 = new Node{50, nullptr};
    Node *node2 = new Node{60, nullptr};
    Node *node3 = new Node{70, nullptr};

    head = node1;

    node1->next = node2;
    node2->next = node3;
    node3->next = nullptr;

    insertFront(30);
    insertFront(20);
    insertFront(10);

    insertBack(80);

    Node *temp = head;
    while (temp != nullptr)
    {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "Null\n";
}
```

2. **Output**

```
10 -> 20 -> 30 -> 50 -> 60 -> 70 -> 80 -> Null
```

3. **Penjelasan**

Program tersebut membuat linked list awal 50 → 60 → 70, kemudian menambahkan 10, 20, dan 30 di bagian depan serta 80 di bagian belakang menggunakan fungsi insertFront dan insertBack, sehingga hasil akhirnya menjadi 10 → 20 → 30 → 50 → 60 → 70 → 80, lalu menampilkan seluruh isi list hingga nullptr

Latihan 4

1. **Code**

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *next;
};

Node *head = nullptr;

void insertBack(int value)
{
    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = nullptr;
    if (head == nullptr)
    {
        head = newNode;
        return;
    }
    Node *temp = head;
    while (temp->next != nullptr)
        temp = temp->next;
    temp->next = newNode;
}

void insertFront(int value)
{
    Node *newNode = new Node;
    newNode->data = value;
```

```
    newNode->next = head;
    head = newNode;
}

void insertAfter(int key, int value)
{
    Node *temp = head;
    while (temp != nullptr && temp->data != key)
        temp = temp->next;
    if (temp == nullptr)
        return;
    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = temp->next;
    temp->next = newNode;
}
void insertBefore(int key, int value)
{
    if (head == nullptr)
        return;
    if (head->data == key)
    {
        insertFront(value);
        return;
    }
    Node *prev = nullptr;
    Node *curr = head;
    while (curr != nullptr && curr->data != key)
    {
        prev = curr;
        curr = curr->next;
    }
    if (curr == nullptr)
        return;
    Node *newNode = new Node;
    newNode->data = value;
    prev->next = newNode;
    newNode->next = curr;
}

int main()
{
    Node *node1 = new Node{50, nullptr};
    Node *node2 = new Node{60, nullptr};
    Node *node3 = new Node{70, nullptr};
```

```
    head = node1;

    node1->next = node2;
    node2->next = node3;

    insertFront(30);
    insertFront(20);
    insertFront(10);

    insertBack(80);
    insertAfter(30, 40);
    insertBefore(60, 55);

    Node *temp = head;
    while (temp != nullptr)
    {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "Null\n";
}
```

2. **Output**

```
10 -> 20 -> 30 -> 40 -> 50 -> 55 -> 60 -> 70 -> 80 -> Null
```

3. **Penjelasan**

Program tersebut membuat linked list awal 50 → 60 → 70, lalu menambahkan 10, 20, dan 30 di depan, 80 di belakang, menyisipkan 40 setelah 30 serta 55 sebelum 60 menggunakan berbagai fungsi insert, sehingga hasil akhirnya menjadi 10 → 20 → 30 → 40 → 50 → 55 → 60 → 70 → 80, kemudian menampilkan seluruh isi list hingga nullptr

Latihan 5

1. **Code**

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *next;
```

```
};

Node *head = nullptr;

void insertBack(int value)
{
    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = nullptr;
    if (head == nullptr)
    {
        head = newNode;
        return;
    }
    Node *temp = head;
    while (temp->next != nullptr)
        temp = temp->next;
    temp->next = newNode;
}

void insertFront(int value)
{
    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = head;
    head = newNode;
}

void insertAfter(int key, int value)
{
    Node *temp = head;
    while (temp != nullptr && temp->data != key)
        temp = temp->next;
    if (temp == nullptr)
        return;
    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = temp->next;
    temp->next = newNode;
}
void insertBefore(int key, int value)
{
    if (head == nullptr)
        return;
    if (head->data == key)
```

```
    {
        insertFront(value);
        return;
    }
    Node *prev = nullptr;
    Node *curr = head;
    while (curr != nullptr && curr->data != key)
    {
        prev = curr;
        curr = curr->next;
    }
    if (curr == nullptr)
        return;
    Node *newNode = new Node;
    newNode->data = value;
    prev->next = newNode;
    newNode->next = curr;
}

void deleteByValue(int key)
{
    if (head == nullptr)
    {
        cout << "List kosong.\n";
        return;
    }
    // Jika head adalah key
    if (head->data == key)
    {
        Node *temp = head;
        head = head->next;
        delete temp;
        cout << "Node dengan nilai " << key << " dihapus.\n";
        return;
    }
    Node *prev = head;
    Node *curr = head->next;
    while (curr != nullptr && curr->data != key)
    {
        prev = curr;
        curr = curr->next;
    }
    if (curr == nullptr)
    {
        cout << "Nilai tidak ditemukan.\n";
```

```
        return;
    }
    prev->next = curr->next;
    delete curr;
    cout << "Node dengan nilai " << key << " dihapus.\n";
}

int main()
{
    Node *node1 = new Node{50, nullptr};
    Node *node2 = new Node{60, nullptr};
    Node *node3 = new Node{70, nullptr};

    head = node1;

    node1->next = node2;
    node2->next = node3;
    node3->next = nullptr;

    insertFront(30);
    insertFront(20);
    insertFront(10);

    insertBack(80);
    insertAfter(30, 40);
    insertBefore(60, 55);
    deleteByValue(80);

    Node *temp = head;
    while (temp != nullptr)
    {
        cout << temp->data << " -> ";
        temp = temp->next;
    }

    cout << "Null\n";
}
```

2. **Output**

```
Node dengan nilai 80 dihapus.
10 -> 20 -> 30 -> 40 -> 50 -> 55 -> 60 -> 70 -> Null
PS F:\DokumenKuliah\Semester 2\Algoritma\Pertemuan 2>
```

3. Penjelasan

Program tersebut membuat linked list awal 50 → 60 → 70, kemudian menambahkan beberapa node di depan, belakang, sebelum, dan sesudah nilai tertentu, lalu menghapus node bernilai 80 menggunakan deleteByValue, sehingga hasil akhirnya menjadi 10 → 20 → 30 → 40 → 50 → 55 → 60 → 70, dan akhirnya menampilkan seluruh isi list hingga nullptr

Tugas

1. Code

```
#include <iostream>

using namespace std;

struct EnemyData
{
    string enemy_name;
    int enemy_health;
    int enemy_attack;
};

struct Node
{
    EnemyData data;
    Node *next;
};

Node *head = nullptr;

int countEnemies()
{
    int count = 0;
    Node *temp = head;

    while (temp != nullptr)
    {
        count++;
        temp = temp->next;
    }
    return count;
}
```

```
void printList()
{
    Node *temp = head;
    while (temp != nullptr)
    {
        cout << temp->data.enemy_name << " (HP:"
             << temp->data.enemy_health << ", ATK:"
             << temp->data.enemy_attack << ") -> ";
        temp = temp->next;
    }
    cout << "Null\n";
}

void insertBack(const EnemyData &value)
{
    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = nullptr;

    if (head == nullptr)
    {
        head = newNode;
        return;
    }

    Node *temp = head;
    while (temp->next != nullptr)
        temp = temp->next;

    temp->next = newNode;
}

void insertFront(const EnemyData &value)
{
    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = head;
    head = newNode;
}

void insertAfter(const string &key, const EnemyData &value)
{
    Node *temp = head;
```

```
    while (temp != nullptr && temp->data.enemy_name != key)
        temp = temp->next;

    if (temp == nullptr)
        return;

    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = temp->next;
    temp->next = newNode;
}

void insertBefore(const string &key, const EnemyData &value)
{
    if (head == nullptr)
        return;

    if (head->data.enemy_name == key)
    {
        insertFront(value);
        return;
    }

    Node *prev = nullptr;
    Node *curr = head;

    while (curr != nullptr && curr->data.enemy_name != key)
    {
        prev = curr;
        curr = curr->next;
    }

    if (curr == nullptr)
        return;

    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = curr;
    prev->next = newNode;
}

void deleteByName(const string &key)
{
    if (head == nullptr)
    {
```

```
        cout << "List kosong.\n";
        return;
    }

    if (head->data.enemy_name == key)
    {
        Node *temp = head;
        head = head->next;
        delete temp;
        cout << "Enemy " << key << " dihapus.\n";
        return;
    }

    Node *prev = head;
    Node *curr = head->next;

    while (curr != nullptr && curr->data.enemy_name != key)
    {
        prev = curr;
        curr = curr->next;
    }

    if (curr == nullptr)
    {
        cout << "Enemy tidak ditemukan.\n";
        return;
    }

    prev->next = curr->next;
    delete curr;
    cout << "Enemy " << key << " dihapus.\n";
}

int main()
{
    int choice;

    while (true)
    {
        cout << "\n===== Enemy Manager =====\n";
        cout << "1. Spawn Enemy (Insert Back)\n";
        cout << "2. Insert Enemy BEFORE target (by name)\n";
        cout << "3. Kill Enemy (Delete by Name)\n";
        cout << "4. Show Enemy list\n";
        cout << "5. Count Enemies\n";
```

```
        cout << "0. Exit\n";
        cout << "Pilih: ";
        cin >> choice;

        cin.ignore();

        switch (choice)
        {
        case 1:
        {
            cout << "======== Spawn Enemy ========\n";

            EnemyData e;
            cout << "Enemy Name: ";
            getline(cin, e.enemy_name);

            cout << "Enemy Health: ";
            cin >> e.enemy_health;

            cout << "Enemy Attack: ";
            cin >> e.enemy_attack;
            cin.ignore();

            insertBack(e);
            break;
        }

        case 2:
        {
            cout << "======== Insert BEFORE ========\n";

            string key;
            cout << "Insert before enemy name: ";
            getline(cin, key);

            EnemyData e;
            cout << "New Enemy Name: ";
            getline(cin, e.enemy_name);

            cout << "Enemy Health: ";
            cin >> e.enemy_health;

            cout << "Enemy Attack: ";
            cin >> e.enemy_attack;
            cin.ignore();
```

```
            insertBefore(key, e);
            break;
        }

        case 3:
        {
            cout << "======== Kill Enemy ========\n";
            cout << "List Enemy yang ada" << endl;
            printList();
            string key;
            cout << "Enemy name to delete: ";
            getline(cin, key);

            deleteByName(key);
            break;
        }

        case 4:
            printList();
            break;

        case 5:
            cout << "Total Enemies: " << countEnemies() << endl;
            break;

        case 0:
            cout << "Keluar...\n";
            return 0;

        default:
            cout << "Pilihan tidak valid\n";
        }
    }
}
```

2. Output

```
===== Enemy Manager =====
1. Spawn Enemy (Insert Back)
2. Insert Enemy BEFORE target (by name)
3. Kill Enemy (Delete by Name)
4. Show Enemy list
5. Count Enemies
0. Exit
Pilih: 1
======== Spawn Enemy ========
Enemy Name: Goblin
Enemy Health: 100
Enemy Attack: 10

===== Enemy Manager =====
1. Spawn Enemy (Insert Back)
2. Insert Enemy BEFORE target (by name)
3. Kill Enemy (Delete by Name)
4. Show Enemy list
5. Count Enemies
0. Exit
Pilih: 4
Goblin (HP:100, ATK:10) -> Null

===== Enemy Manager =====
1. Spawn Enemy (Insert Back)
2. Insert Enemy BEFORE target (by name)
3. Kill Enemy (Delete by Name)
4. Show Enemy list
5. Count Enemies
0. Exit
Pilih:
```

3. Penjelasan

Program tersebut membuat sistem game sederhana berbasis linked list untuk mengelola musuh, di mana pemain dapat men-spawn enemy (tambah di belakang), menyisipkan enemy sebelum target tertentu (misalnya boss), menghapus enemy yang mati berdasarkan nama, menampilkan semua enemy aktif, serta menghitung total enemy